

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re C D E

Algorithmique : structures de contrôle, fonctions et procédures

Module 2 : Algorithmique et programmation

Consigne : Les exercices suivants portent sur la conception modulaire d'algorithmes, la manipulation de tableaux, la gestion de la portée des variables et les modes de passage de paramètres. Toute trace d'exécution doit être rigoureuse et justifiée.

EXERCICE 1 Tri modulaire et analyse de complexité

On souhaite trier un tableau T de N entiers ($N \geq 2$) par l'algorithme du tri par sélection (Selection Sort). On impose une décomposition modulaire stricte.

1. Écrire une fonction `IndiceMin(T, debut, fin)` qui retourne l'indice de la plus petite valeur dans T entre les indices `debut` et `fin` (inclus). Préciser le type de retour et le mode de passage du tableau (justifier).
2. Écrire une procédure `Permuter(var a, var b : Entier)` échangeant le contenu de deux variables. Justifier le mode de passage choisi.
3. Écrire la procédure principale `TrierSelection(var T : Tableau[1..N] d'entiers)` utilisant les deux sous-programmes ci-dessus.
4. Analyse critique : On déclare une variable globale `CompteurComparaisons : Entier` initialisée à 0 avant l'appel à `TrierSelection`. La fonction `IndiceMin` incrémente ce compteur à chaque comparaison.
 - (a) Quelle est la valeur finale de `CompteurComparaisons` en fonction de N ? Démontrer.
 - (b) Si l'on déclare `CompteurComparaisons` comme variable locale à `TrierSelection` et qu'on la passe en paramètre à `IndiceMin`, quel mode de passage faut-il choisir? Réécrire l'en-tête de `IndiceMin` en conséquence.

EXERCICE 2 Calcul numérique et structures de données : Suite de Fibonacci

Soit la suite de Fibonacci définie par $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$. On veut stocker les $N + 1$ premiers termes ($N \geq 20$) dans un tableau `Fib[0..N]`.

1. Proposer un algorithme itératif (procédure `RemplirIteratif`) remplissant le tableau. Analyser sa complexité temporelle et spatiale.
2. On propose une version récursive avec mémorisation (programmation dynamique) :

```
Fonction FibMemo(n : Entier) : Entier
    Si Fib[n] <> -1 Alors Renvoyer Fib[n]
    Sinon
        Fib[n] <- FibMemo(n-1) + FibMemo(n-2)
```

```

    Renvoyer Fib[n]
  FinSi
FinFonction

```

Le tableau `Fib` est global, initialisé avec `Fib[0]=0`, `Fib[1]=1` et `-1` ailleurs.

- (a) Expliquer pourquoi l'appel `FibMemo(N)` remplit correctement le tableau. Quel est l'avantage majeur par rapport à la récursion naïve sans tableau?
 - (b) Portée et effets de bord : Si l'on déclare `Fib` comme variable locale à une procédure `CalculerFib(N)` et qu'on la passe à `FibMemo`, quel mode de passage est indispensable? Que se passe-t-il si l'on choisit le passage par valeur pour le tableau?
3. Problème ouvert : Les entiers standards (32 bits signés) débordent pour F_{47} . Proposer une structure de données (tableau d'entiers) pour stocker un grand entier chiffre par chiffre (base 10 000 par exemple) et décrire l'algorithme d'addition de deux tels "grands entiers" sous forme de procédure `GrandAddition(var A, B, Res : TableauEntiers)`. On ne demande pas le code complet, mais la logique de parcours et de gestion de la retenue.

EXERCICE 3 Implantation d'une Pile (Stack) par tableau et gestion d'erreurs

On implémente une pile d'entiers de capacité maximale `MAX` à l'aide d'un tableau `P[1..MAX]` et d'un entier `Sommet` (0 si pile vide). On impose l'encapsulation : `P` et `Sommet` sont des variables globales au module "Pile".

1. Écrire les procédures `Empiler(valeur : Entier)` et `Depiler(var valeur : Entier)`. Elles doivent gérer les erreurs (dépassement de capacité, pile vide) en positionnant une variable globale `Erreur : Booleen` (True si erreur) sans arrêter le programme.
2. Écrire une fonction `SommetPile() : Entier` qui retourne la valeur au sommet sans dépiler (retourne 0 si vide, positionne `Erreur`).
3. Application : Conversion base 10 → base 2. Écrire un algorithme principal qui lit un entier positif X , utilise la pile pour stocker les remainders des divisions successives par 2, puis dépile pour afficher l'écriture binaire. Utiliser uniquement les procédures/fonctions définies aux questions 1 et 2 (interdiction d'accéder directement à `P` ou `Sommet` dans le principal).
4. Analyse de la modularité : On souhaite maintenant gérer deux piles indépendantes simultanément (ex : `PileA` et `PileB`) avec le même code de procédures.
 - (a) Expliquer pourquoi l'approche "variables globales" de la question 1 empêche cette réutilisation sans duplication de code.
 - (b) Proposer une nouvelle déclaration de type `TypePile` (enregistrement/structure) et réécrire l'en-tête de `Empiler` pour qu'elle soit générique (utilisable pour n'im-

porte quelle instance de pile). Préciser le mode de passage du paramètre pile.

EXERCICE 4 Traçage complexe : Variables globales, locales et passages de paramètres

On considère l'algorithme suivant (syntaxe proche Pascal/Algobox). A et B sont des variables globales entières.

Variables Globales

A, B : Entier

Debut

A <- 10

B <- 5

Appel Proc1(A, B)

Afficher "Fin Principal : A=", A, " B=", B

Fin

Procedure Proc1(X : Entier; var Y : Entier)

Variable Locale

Z : Entier

Debut

Z <- X + Y

X <- X * 2

Y <- Y - 1

Si Z > 10 Alors

Appel Proc2(Z, Y)

FinSi

Afficher "Fin Proc1 : X=", X, " Y=", Y, " Z=", Z

Fin

Procedure Proc2(var U : Entier; V : Entier)

Debut

A <- A + U // Accès variable globale A

U <- U + V

V <- V * 2

Afficher "Fin Proc2 : U=", U, " V=", V, " A(glob)=", A

Fin

1. Effectuer la trace complète d'exécution de cet algorithme en remplissant un tableau de trace (Colonnes : Instruction exécutée, A, B, X, Y, Z, U, V, Sortie écran). Noter "N/A" si variable non visible.
2. Justifier précisément les valeurs finales de A et B affichées par le programme principal en lien avec les modes de passage (valeur / variable) des paramètres de Proc1 et Proc2.

3. Quel aurait été l'affichage final de A et B si le paramètre X de Proc1 avait été passé par variable (var X)? Justifier sans refaire toute la trace.